

實驗結果整理：Rust Concurrency Ticket Platform Demo

1. 實驗目的

本實驗以「高併發演唱會售票平台」作為案例，觀察 Rust 在並行程式設計中的兩種不同工具：

- `std::sync::mpsc`：用於即時訂單處理，展示 message passing、bounded queue 與 backpressure。
- Rayon：用於售後批次分析，展示資料平行處理與 sequential / parallel 效能比較。

本實驗的重點不是單純比較哪一個工具比較快，而是說明不同並行工具適合解決不同類型的軟體工程問題。

2. 實驗環境與共同設定

Rust 程式皆使用 release mode 執行：

```
cargo run --release -- ...
```

共同售票設定如下：

參數	數值	含意
<code>tickets</code>	100	初始庫存票數
<code>producers</code>	16	同時送出訂單的前端節點或售票窗口數
<code>orders</code>	80	每個 producer 送出的訂單數
<code>total_orders</code>	1280	總訂單數，等於 <code>16 * 80</code>
<code>queue_capacity</code>	32	<code>mpsc</code> bounded channel 容量
<code>service-delay-us</code>	500	中央售票服務處理每筆訂單的模擬延遲

系統需要維持的 business invariant：

```
sold + remaining == initial_inventory  
sold + rejected == submitted_orders
```

第一個 invariant 用來確認沒有超賣；第二個 invariant 用來確認所有訂單最後都被歸類為成功售出或售罄拒絕。

3. 實驗一：mpsc 即時訂單服務

3.1 執行指令

```
cargo run --release -- mpsc --tickets 100 --producers 16 --orders 80 --queue-  
capacity 32 --service-delay-us 500
```

3.2 實驗數據

指標	數值
初始票數	100
producer 數量	16
每個 producer 訂單數	80
總訂單數	1280
queue capacity	32
每筆訂單處理延遲	500 us
submitted orders	1280
producer submitted count	1280
sold	100
rejected / sold out	1180
regular sold	80
VIP sold	20
regular remaining	0
VIP remaining	0
invariant 是否成立	true
producer sends delayed by backpressure	1248
cumulative producer send wait	20.185277s
elapsed	1.3359624s

3.3 步驟含意

此實驗模擬 16 個前端節點同時送出訂單，所有訂單先進入 bounded `mpsc` channel，再由單一中央售票服務依序處理庫存。

```
producer threads -> bounded mpsc channel -> central ticket office
```

這個設計的核心是「讓中央售票服務擁有庫存」。producer 不直接修改庫存，因此系統不需要讓多個 thread 同時讀寫同一份票數。這和單純用多個 thread 共同扣同一個 counter 的設計不同，`mpsc` 把共享可變狀態轉換成訊息傳遞與所有權邊界。

3.4 觀察重點

- `sold = 100` 且 `remaining = 0`，表示所有庫存都被售出，但沒有超過 100 張。
- `rejected / sold out = 1180`，表示剩下的訂單被明確歸類為售罄拒絕，而不是造成錯誤售票。
- `sold + remaining == initial` 成立，代表沒有超賣。
- `sold + rejected == submitted` 成立，代表所有訂單都有被處理。
- `producer sends delayed by backpressure = 1248`，表示大多數送出動作都曾因 queue 容量有限或中央服務處理速度較慢而等待。
- `cumulative producer send wait = 20.185277s` 大於整體 `elapsed = 1.3359624s`，是因為多個 producer thread 的等待時間是累加值；不同 thread 的等待會在真實時間中重疊發生。

4. 實驗二：Rayon 批次分析，1,000,000 筆資料

4.1 執行指令

```
cargo run --release -- rayon --analytics-records 1000000 --producers 16
```

4.2 實驗數據

指標	數值
analytics records	1,000,000
sequential elapsed	101.6667ms
parallel elapsed	15.4698ms
speedup	6.57x
sequential result equals parallel result	true
revenue	\$16,714,463.00
regular sales	857,136
VIP sales	142,864
high value sales	142,864
fraud / manual review candidates	20,117
busiest source	#15
busiest source sales	62,500
checksum	5346

4.3 步驟含意

此實驗模擬售票結束後的批次資料分析。每一筆銷售紀錄都可以被獨立處理，例如計算營收、票種數量、高價票數、人工審查候選訂單與來源統計。

Rayon 使用資料平行的概念，將原本 sequential 的迭代分析拆分到多個 CPU core 上執行：

```
sales records -> par_iter -> fold -> reduce -> analytics result
```

此處 Rayon 的重點是加速 CPU-bound batch job，而不是處理即時訂單流程或共享庫存扣減。

4.4 觀察重點

- sequential 與 parallel 結果完全一致，表示平行化沒有改變分析結果。
- 1,000,000 筆資料下，Rayon parallel elapsed 為 15.4698ms，明顯低於 sequential 的 101.6667ms。
- speedup 達 6.57x，代表資料量足夠時，Rayon 可以有效利用多核心 CPU。
- busiest source 為 #15，且銷售數為 62,500，剛好是 1,000,000 / 16，表示資料平均分散到 16 個來源。

5. 實驗三：Rayon 批次分析，2,000,000 筆資料

5.1 執行指令

```
cargo run --release -- rayon --analytics-records 2000000 --producers 16
```

5.2 實驗數據

指標	數值
analytics records	2,000,000
sequential elapsed	201.5869ms
parallel elapsed	25.6605ms
speedup	7.86x
sequential result equals parallel result	true
revenue	\$33,428,926.25
regular sales	1,714,272
VIP sales	285,728
high value sales	285,728
fraud / manual review candidates	39,912
busiest source	#15
busiest source sales	125,000
checksum	25158103

5.3 觀察重點

- 資料量從 1,000,000 筆增加到 2,000,000 筆後，sequential elapsed 從 101.6667ms 增加到 201.5869ms，幾乎接近線性成長。
- parallel elapsed 從 15.4698ms 增加到 25.6605ms，雖然也增加，但增加幅度較小。
- speedup 從 6.57x 提升到 7.86x，表示資料量越大，Rayon 的平行化成本越容易被攤平。
- sequential result equals parallel result 仍為 true，代表平行化後仍保持計算正確性。

6. 實驗四：完整 all 流程

6.1 執行指令

```
cargo run --release -- all --tickets 100 --producers 16 --orders 80 --queue-capacity 32 --analytics-records 500000
```

6.2 mpsc 即時訂單服務數據

指標	數值
submitted orders	1280
producer submitted count	1280
sold	100
rejected / sold out	1180
regular sold	80
VIP sold	20
regular remaining	0
VIP remaining	0
invariant 是否成立	true
producer sends delayed by backpressure	1247
cumulative producer send wait	20.0003366s
elapsed	1.3245338s

6.3 Rayon 批次分析數據

指標	數值
analytics records	500,000
sequential elapsed	59.0432ms
parallel elapsed	16.9455ms
speedup	3.48x
sequential result equals parallel result	true

指標	數值
revenue	\$8,357,442.50
regular sales	428,556
VIP sales	71,444
high value sales	71,444
fraud / manual review candidates	10,096
busiest source	#0
busiest source sales	31,279
checksum	3316827

6.4 步驟含意

a11 模式把整個平台 demo 串起來。Stage 1 提醒先執行 C race-condition counter，Stage 2 執行 Rust **mpsc** 即時訂單服務，Stage 3 再用銷售紀錄作為售後分析資料來源，執行 Rayon 批次分析。

這個流程能清楚區分：

- C race-condition counter：展示錯誤的 shared mutable state 可能破壞售票需求。
- Rust **mpsc**：展示即時訂單處理與 backpressure。
- Rayon：展示大量售後資料的平行分析。

7. Rayon 資料量比較

資料筆數	Sequential	Parallel	Speedup
500,000	59.0432ms	16.9455ms	3.48x
1,000,000	101.6667ms	15.4698ms	6.57x
2,000,000	201.5869ms	25.6605ms	7.86x

觀察：

- Sequential 執行時間隨資料量增加而明顯上升。
- Parallel 執行時間也會上升，但上升幅度相對較小。
- 資料量越大，Rayon 的 thread pool、工作切分與 reduce 成本越容易被大量運算攤平，因此 speedup 更明顯。
- 500,000 筆時 speedup 為 3.48x，2,000,000 筆時提升到 7.86x，代表 Rayon 更適合有足夠資料量的 CPU-bound 批次工作。

8. 實驗結論

本實驗可以得到以下結論：

1. **mpsc** 適合用於即時系統中的訊息傳遞。

在售票平台中，多個 producer 不直接修改庫存，而是將訂單送進 bounded channel，由單一中央售票服務擁有並修改庫存。這種設計能把共享可變狀態集中在清楚的 ownership boundary 內，避免多個 thread 同時扣同一份票數。

2. Bounded channel 可以呈現 backpressure。

實驗中 `producer sends delayed by backpressure` 超過 1200 次，表示當中央售票服務處理速度有限、queue 容量只有 32 時，producer 會被迫等待。這反映真實高併發系統中的流量控制問題：系統不只是要能接收請求，也要能在下游處理速度不足時控制壓力。

3. Rayon 適合售後批次分析，而不是即時扣庫存。

Rayon 的強項在於將大量彼此獨立的資料處理平行化。本實驗中 revenue、票種統計、異常候選訂單與來源統計都能用 `par_iter`、`fold`、`reduce` 處理，且 parallel result 與 sequential result 完全一致。

4. 資料量越大，Rayon 效益越明顯。

1,000,000 筆資料時 speedup 為 6.57x，2,000,000 筆資料時 speedup 提升到 7.86x。這表示當資料量足夠大時，平行化成本會被攤平，多核心 CPU 的優勢更容易被觀察到。

5. `mpsc` 與 Rayon 都屬於並行程式設計工具，但解決的問題不同。

`mpsc` 解決的是 live task flow、message passing 與 ownership boundary；Rayon 解決的是 batch data parallelism。將兩者放在同一個售票平台的不同子系統中，比把兩者硬套在同一個扣票流程中更符合軟體工程實務。

總結來說，本專案展示了 Rust 並行程式設計的兩種重要面向：使用 `mpsc` 設計穩定的即時訂單流程，以及使用 Rayon 加速大量資料的售後分析。這兩者共同說明 Rust 不只是提供多執行緒語法，也鼓勵開發者用更清楚的 ownership 與資料流設計來降低 race condition 風險。